

中国矿业大学计算机学院

## Linux 操作系统课程设计报告

课程名称：Linux 操作系统

报告时间：2025.11.18

学生姓名：王志豪

学    号：08232337

专    业：数据科学与大数据技术

任课教师：李昕

2025 年 11 月

# 目录

|                                                           |           |
|-----------------------------------------------------------|-----------|
| <b>1 摘要</b>                                               | <b>3</b>  |
| <b>2 引言</b>                                               | <b>3</b>  |
| <b>3 需求分析</b>                                             | <b>4</b>  |
| <b>4 系统总体设计</b>                                           | <b>5</b>  |
| 4.1 系统架构 . . . . .                                        | 5         |
| 4.2 数据流向 . . . . .                                        | 6         |
| <b>5 模块设计</b>                                             | <b>10</b> |
| 5.1 主要数据结构 . . . . .                                      | 10        |
| 5.1.1 协议与基础结构 (public.h) . . . . .                        | 10        |
| 5.1.2 聊天消息模块 (message.hpp) . . . . .                      | 10        |
| 5.1.3 缓存模块 (MCache.h) . . . . .                           | 10        |
| 5.1.4 线程池模块 (tPool.h) . . . . .                           | 10        |
| 5.1.5 聊天服务器与客户端模块 (websocket-client/server.hpp) . . . . . | 11        |
| 5.1.6 文件传输服务器与客户端模块 (fileTrans) . . . . .                 | 11        |
| 5.1.7 连接管理服务器与客户端模块 (conSer 和 conCli) . . . . .           | 11        |
| 5.2 登录注册前后端模块设计 . . . . .                                 | 11        |
| 5.3 文件传输前后端模块设计 . . . . .                                 | 13        |
| 5.4 实时聊天前后端模块设计 . . . . .                                 | 16        |
| <b>6 功能展示与分析</b>                                          | <b>17</b> |
| 6.1 登录注册功能分析 . . . . .                                    | 17        |
| 6.2 文件传输功能分析 . . . . .                                    | 18        |
| 6.2.1 文件上传 . . . . .                                      | 18        |
| 6.2.2 文件下载 . . . . .                                      | 20        |
| 6.2.3 查看文件 . . . . .                                      | 21        |
| 6.3 实时聊天功能分析 . . . . .                                    | 21        |
| <b>7 开发环境与项目结构</b>                                        | <b>22</b> |
| <b>8 使用说明</b>                                             | <b>24</b> |
| 8.1 编译方式 . . . . .                                        | 24        |
| 8.2 数据库与缓存准备 . . . . .                                    | 24        |
| 8.3 运行方式 . . . . .                                        | 25        |

|       |    |
|-------|----|
| 目录    | 2  |
| 9 总结  | 26 |
| 10 附录 | 27 |

## 评分标准

|              | 标准                                                  | 分项得分 | 总分 |
|--------------|-----------------------------------------------------|------|----|
| Linux 编程综合应用 | 题目的新颖性，功能的完备性，正确性（60 分）                             |      |    |
|              | 实验运行结果截图，且对实验结果进行深入分析（20 分）                         |      |    |
|              | 关键代码要注释（10 分）                                       |      |    |
|              | 报告格式规范，有目录，提交电子版报告与源代码，连同两次作业放在一个压缩包里。打印纸质版报告（10 分） |      |    |

## 1 摘要

本项目针对现代网络应用在大规模数据存储、安全传输、跨终端访问与高并发交互方面的实际需求，设计并实现了一套运行于 Linux 平台、以 C++ 为主要开发语言的云盘系统原型。系统以分层式 C/S 架构为总体设计框架，通过客户端与服务端职责分离方式实现结构清晰、可扩展性强的云服务模型。客户端承担用户命令解析、认证交互、文件操作请求与结果展示等前端逻辑；服务端则作为统一的业务中心，集成高并发线程池、任务队列、文件管理模块、聊天模块与数据库访问模块，负责调度处理所有来自客户端的请求，并执行身份校验、数据读写与业务逻辑管理。

通信方面，系统基于 TCP socket 构建稳定连接，通过自定义二进制传输协议对报文进行结构化封装，确保跨平台通信的高效性与可靠性。核心业务功能包括用户注册/登录、云盘文件上传下载、断点续传、文件目录与元数据管理、实时聊天消息收发等。每项功能均采用模块化、接口化的方式实现，在系统内部形成清晰的功能边界，便于后续维护、扩展与版本迭代。

在并发架构方面，服务端采用典型的生产者-消费者模型构建线程池，通过任务队列实现任务分发与负载均衡，有效提升多用户同时访问时的整体吞吐能力。数据库层面引入连接池机制，复用数据库连接资源，降低连接创建开销，并确保数据存取的一致性与事务安全。文件传输模块使用 MD5 校验实现数据完整性验证，能够在断点续传与大文件传输场景下保证文件内容可靠。系统还设计了多级日志记录机制，对用户操作、网络事件、异常情况等进行全流程追踪，使服务具备较强的可监控性与可恢复性。

整体来看，本项目构建了一套具备高可靠性、可扩展性与工程可维护性的云盘原型系统，在通信协议、并发模型、模块划分、安全校验与数据存储等方面均采用工程实践中可复用的设计方式。系统能够支持多终端并发访问、模块独立升级及横向扩展，为进一步演化为生产级云存储服务奠定了坚实基础。

**关键词：** Linux；云盘系统；多进程；TCP socket；MD5；生产者-消费者模型

## 2 引言

随着网络环境的不断发展，用户对跨设备文件管理、数据备份以及在线沟通的需求日益增加。传统的本地存储方式在安全性、可访问性和多设备同步方面存在明显限制，而现有云盘服务虽然成熟，但其实现机制、系统架构及并发处理策略通常较为复杂，不利于学习与实验性验证。在这种情况下，构建一套功能完整、结构清晰、具备可扩展性的云盘与即时通讯系统，既能够满足基本的文件存储需求，也能为网络编程、并发处理和系统设计等方向提供实践平台。

本项目围绕上述目标，设计并实现了一个基于 C/S 架构的云盘系统 CloudPan。系统整体由客户端、服务端和数据存储层构成，客户端负责用户登录、文件管理和消息交互等操作，服务端承担请求接入、业务调度和任务处理，数据层则用于维护文件实体、

用户信息以及聊天记录。项目实现过程中，重点考虑了并发访问、数据一致性和网络通信效率等因素，在服务端引入线程池与数据库连接池，以提升多用户场景下的响应性能；在通信机制上采用 TCP socket 与自定义协议，以确保传输过程中的稳定性和可控性；在数据安全方面，加入了 MD5 校验、日志记录和异常处理等措施，以增强系统的可靠性。

技术分析报告从架构设计、模块划分、数据流向、关键机制以及整体实现方式等方面，对 CloudPan 系统进行了全面梳理和总结。通过对聊天模块、文件传输模块、数据库模块、线程池模型等关键部分的分析，可以完整呈现系统从请求入口到业务处理、再到数据落盘的全过程。同时，报告对协议设计、安全机制以及扩展性考虑也作了说明，为后续改进、优化或横向扩展提供了参考。

本项目旨在构建一个结构合理、功能完善且易于维护的云盘系统原型，使开发者能够在实践中深入理解网络通信、并发模型、模块化设计和系统工程的核心思想，为进一步的研究与开发奠定基础。

### 3 需求分析

| 类别    | 子项    | 需求内容                                                  |
|-------|-------|-------------------------------------------------------|
| 功能需求  | 用户管理  | 注册、登录认证、异常断线处理。                                       |
|       | 文件管理  | 文件上传下载、删除与重命名、断点续传、MD5 校验、文件元数据记录。                    |
|       | 即时通讯  | 群聊消息转发、消息历史保存与查询、在线状态提示。                              |
|       | 日志与监控 | 操作日志、异常日志、运行状态记录（可扩展）。                                |
| 非功能需求 | 性能    | 支持多客户端并发，线程池与连接池提升吞吐，分片降低大文件传输延迟。                     |
|       | 可靠性   | 断点续传机制、MD5 文件完整性校验、完善的异常处理与恢复能力。                      |
|       | 安全性   | 密码 MD5 加密存储、权限校验、自定义协议格式检查与非法请求过滤。                    |
|       | 可扩展性  | 模块化设计，协议指令可扩展，服务端可通过增加线程或优化存储扩容。                      |
|       | 可维护性  | 代码结构清晰，日志完善，接口统一，便于调试与迭代更新。                           |
| 系统约束  | 技术环境  | C++ 开发、Linux 平台、TCP Socket 通信、自定义协议、SQLite/MySQL 数据库。 |
|       | 资源限制  | 线程池规模受 CPU 影响，大文件传输受网络带宽影响，公网/局域网表现不同。                |
|       | 开发约束  | 客户端与服务端需严格遵守统一协议，任务需适配线程池并发模型，数据库访问依赖连接池。             |

表 1: CloudPan 系统需求分析表

## 4 系统总体设计

### 4.1 系统架构

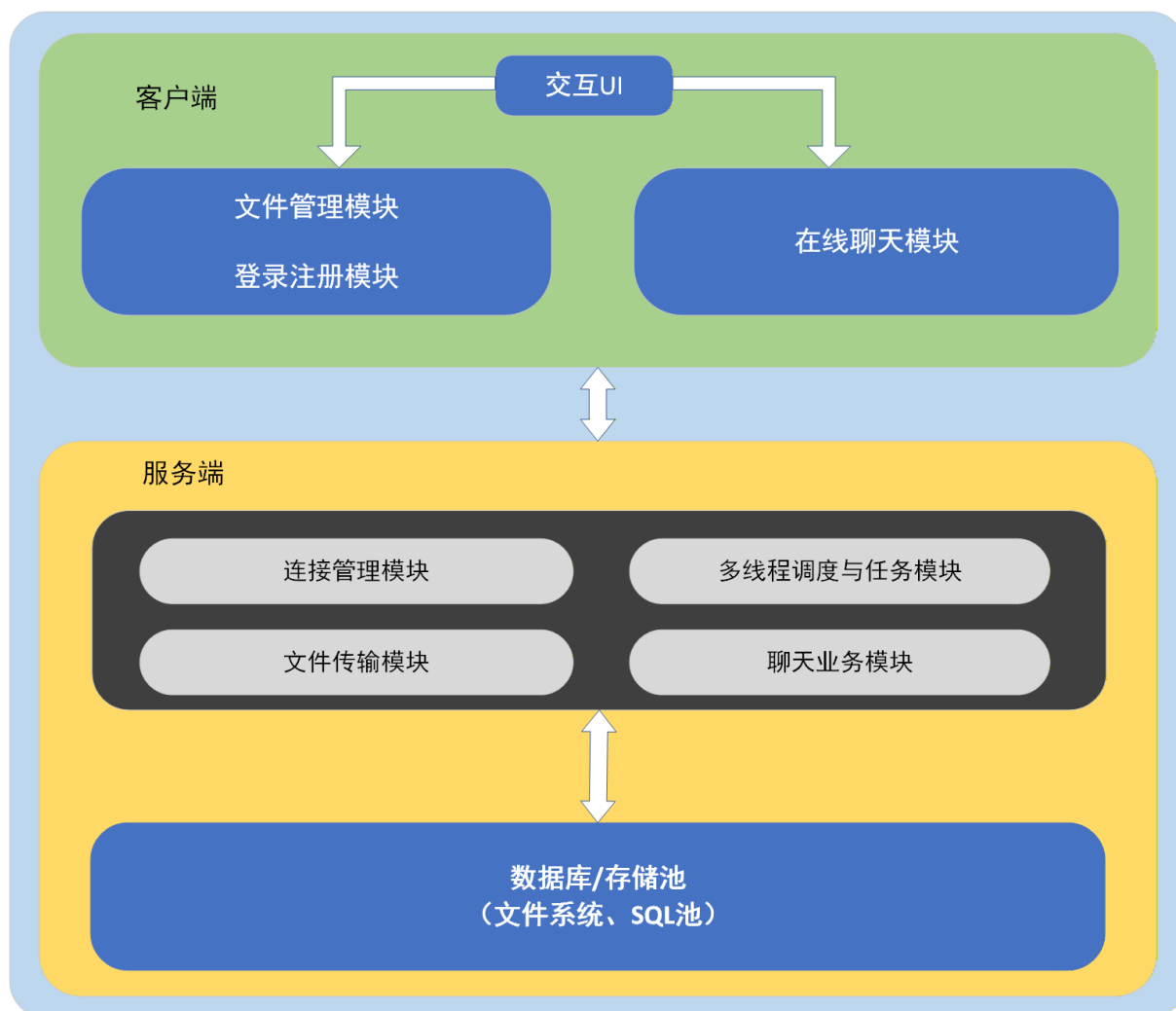


图 1: 系统总体架构示意图

本项目的架构遵循经典的 C/S（客户端/服务器）模式设计，架构主要可以分为三层，分别是客户端、服务端以及存储层，当然，本项目的服务端同时本地部署了数据库以及服务端的业务模块，因此可以将存储层与服务端看成同一层，后续描述依旧延续三层架构模式。

首先，在上层的客户端，用户通过交互面板对操作发出相应的请求。本项目中，由于存在登陆注册校验部分，因此交互面板也分为两层，其中第一层即为用户登录注册阶段，当系统校验用户登录/注册成功后，则会跳转到第二层，也就是真实的操作交互层，该层所提供的服务分为四种，它们分别是：1. 上传文件，2. 下载文件，3. 聊天，4. 查看文件，0. 返回登录界面。用户只需输入相应的数字，即可接收服务端提供的服务。

其次，是服务端，服务端的核心功能模块分为三种，它们分别是：连接管理模块、文件传输模块、聊天业务模块。其中连接管理模块主要负责处理用户注册、登录、认证，

从数据库中读取用户信息，以及向上层提供用户信息查询接口的功能；文件传输模块则主要负责文件上传、下载、删除、重命名，大文件分片上传及断点续传，并且该模块还实现了基于 MD5 的数据完整性校验，文件存储路径管理，更新数据库中的文件元数据的功能，文件的传输与管理设计严谨、功能完善；最后是聊天业务模块，它主要负责客户端消息转发给在线用户，离线消息的缓存与写库聊天记录查询，本模块的聊天业务是广播式的，也就是说，所有用户是共享同一聊天室的（连接同一服务器端口），因此本项目的业务更适用于小规模会员私域服务。并且服务端的设计采用了生产者—消费者模型的线程池作为高并发处理核心，该设计实现了主线程负责接收客户端连接与消息，将解析后的任务投入任务队列，多个工作线程长期阻塞等待队列任务，工作线程从队列中取出任务并执行对应模块逻辑，并且执行完毕后返回响应或写入存储层。这为数据库操作、文件传输等耗时流程提供了稳定的执行环境。

最后是存储层的设计，存储层主要采用 Mysql 数据库进行设计，数据库中主要记录两类信息，分别是：文件的本地存储信息表、用户注册信息表。文件的本地存储信息不仅记录了网盘存储文件的基本信息，还索引了网盘文件在本地存储的文件路径，并且当服务器执行上传任务时，服务器会将用户的存储文件地址分离，并写入数据库中，这样保证了用户间的文件存储不会互相影响，实现了隐私性的保护。当然，当服务端执行下载任务时，也需要访问数据库查询对应用户存储的文件地址，然后去相应的位置读取文件信息；用户的注册信息表首先会在用户成功注册时被更新，当用户尝试登录时，系统会查询数据库，确认用户是否注册过账户，并且校验密码是否正确，并将校验信息返回给前端以通知其是否放行。

## 4.2 数据流向

接下来是关于本项目中核心的三个模块的数据流图：

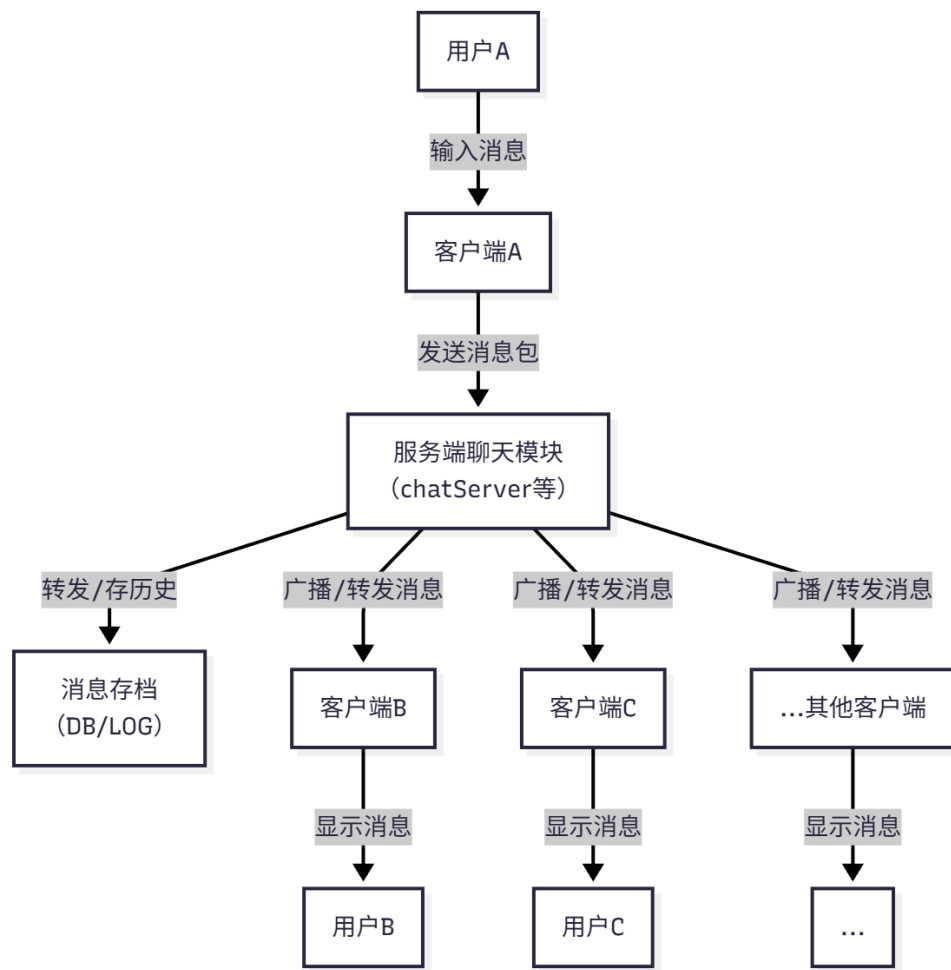


图 2: 聊天模块数据流图

聊天模块数据流图由2可知，该数据流由用户端输入、消息传输与服务端处理协同构成。外部实体包括用户 A 及其他在线用户（如用户 B、C 等）；核心数据流涵盖消息输入、客户端生成的消息协议包、服务端转发包、历史消息存档写入及客户端侧的消息显示。数据存储为聊天消息日志或数据库，用于持久化会话记录。在处理过程中，客户端负责消息的发送与接收；服务端聊天模块承担协议解析、消息存档与转发逻辑。整体流程为：用户 A 在客户端 A 输入聊天内容，客户端将其包装为消息协议包并通过 TCP 提交给服务端聊天模块；服务端解析该消息，依据聊天类型（点对点或群组）确定转发策略，并在需要时将内容写入聊天历史数据库或日志文件；随后，服务端将消息按目标广播或定向发送至其他客户端（如 B、C），客户端在接收后触发界面展示，使用户 B、C 能够查看并继续回复，形成持续的会话循环。



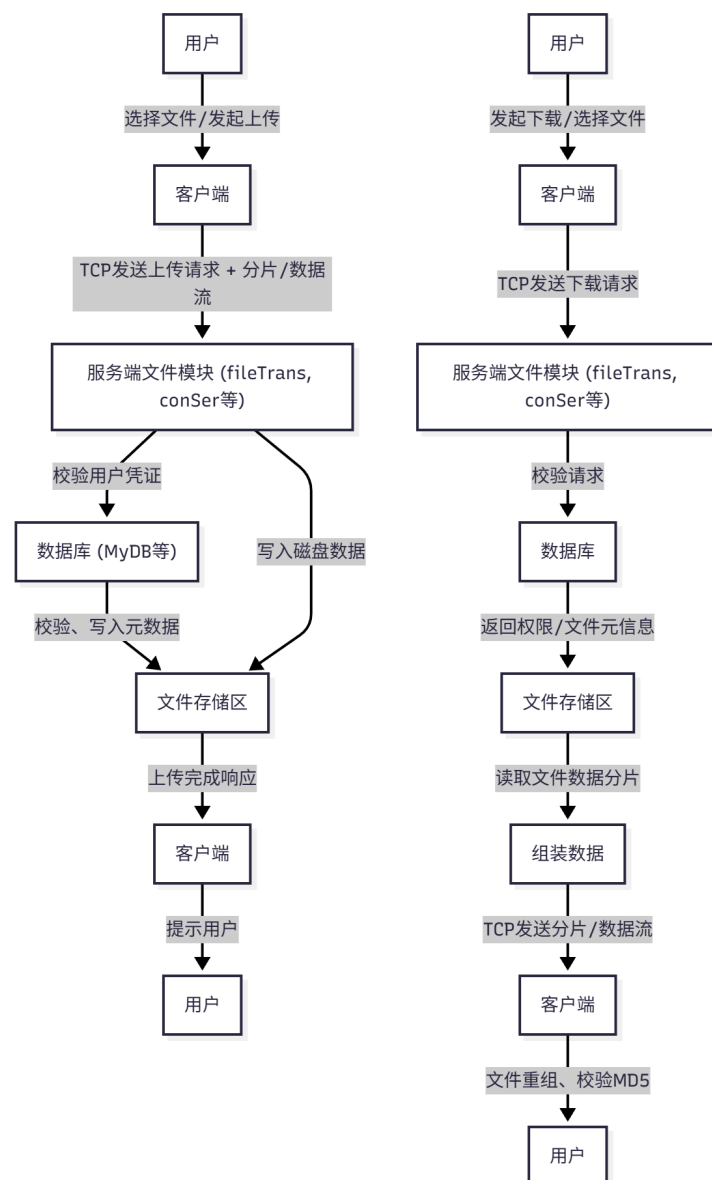


图 3: 文件传输模块数据流图

文件传输模块的数据流图如3所示，该数据流围绕用户操作触发，并由客户端、服务端与存储系统协同完成。外部实体为用户，其在前端发起上传、下载、查询等操作后，客户端负责对请求进行打包或对接收的数据进行拆包处理。请求进入服务端后，首先由鉴权模块执行身份校验与权限验证；验证通过后，服务端根据操作类型与元信息需求与数据库交互，读取或写入文件的相关元数据（如文件名、大小、校验码等），并与底层文件系统执行实际的数据存取操作。随后，服务端将处理结果或数据流封装为响应返回客户端。客户端收到响应后对数据进行重组或校验，并将最终结果反馈给用户界面，使用户能够获得直观的操作反馈或访问相应的文件数据。整个流程确保了从请求产生、鉴权执行、数据存储到结果回传的完整业务闭环。

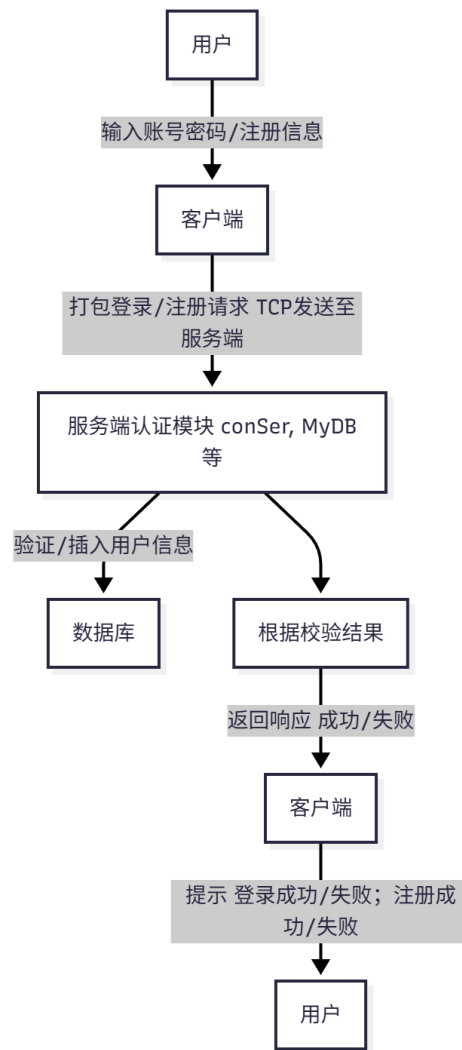


图 4: 登录注册模块数据流图

登录注册模块的数据流图如4所示，在本系统中，用户作为外部实体，通过客户端发起登录或注册操作。用户在客户端输入用户名和密码后，客户端将请求封装成网络数据包并发送至服务端。服务端接收请求后进行解析和处理：对于登录操作，会校验数据库中的用户信息；对于注册操作，则将新用户信息写入数据库。处理完成后，服务端将操作结果（成功或失败）反馈给客户端，客户端随后对用户进行提示，实现完整的用户认证与交互流程。

总而言之，系统的数据流由用户操作触发，通过客户端、服务端和存储系统协同完成形成完整闭环。用户在客户端发起登录注册、聊天或文件操作，客户端将请求封装为网络数据包发送至服务端，服务端按功能模块处理请求——登录注册校验或写入数据库，聊天模块解析、存档并转发消息，文件模块鉴权后与数据库及文件系统交互存取数据——处理完成后将响应返回客户端，由客户端解析并反馈给用户界面。整个数据流在“客户端——服务端 TCP 通信”和“服务端——存储/数据库/日志”之间循环流转，实现高可靠、高一致、高可追踪的业务闭环，各业务虽独立但遵循统一的数据流模式和并发调度策略，确保系统清晰、可扩展且工程可靠。

## 5 模块设计

### 5.1 主要数据结构

#### 5.1.1 协议与基础结构 (public.h)

**OP\_TYPE (枚举)** ENTER, REGISTER, UPLode, DOWNLOAD, LS

**msgInfo (结构体)** type (OP\_TYPE), user\_id (string), password (string)

**fileinfo (结构体)** file\_name、file\_MD5、file\_path、file\_size、file\_chunk\_size、  
chunk\_size、trans\_status

**filedata (结构体)** file\_name、id、offset、trans\_status

#### 5.1.2 聊天消息模块 (message.hpp)

**chat::Message (类)** username、content、timestamp; 方法: toString(), fromString()

#### 5.1.3 缓存模块 (MCache.h)

**MCache (类)** memc (memcached\_st\*), servers (memcached\_server\_st\*),  
rc (memcached\_return); 方法: insertValue(), getValue(),  
deleteKey()

#### 5.1.4 线程池模块 (tPool.h)

**tpool\_work\_t (结构体)** routine (函数指针), arg (void\*), next (tpool\_work\_t\*)

**tpool\_t (结构体)** thr\_id (pthread\_t[]), queue\_head、queue\_tail  
(tpool\_work\_t\*), queue\_lock (pthread\_mutex\_t),  
queue\_ready (pthread\_cond\_t), shutdown (bool); 方法:  
createPool(), addWorkToPool(), destroyPool()

### 5.1.5 聊天服务器与客户端模块 (websocket-client/server.hpp)

**ConnectionHdl** (别名) WebSocket 连接句柄名)

**ChatClient** (类) client (WebSocketClient), connection (ConnectionHdl), username (string), messageCallback (函数), connected (bool)

**ChatServer** (类) server (WebSocketServer), connections (set<ConnectionHdl>), messageCallback (函数), running (bool), port (int)

### 5.1.6 文件传输服务器与客户端模块 (fileTrans)

**FileTrans** (类) trans\_fd, connfd, user\_id, msg (msgInfo), sql (list<string>), m\_fileinfo (fileinfo), m\_filedata (filedata)

### 5.1.7 连接管理服务器与客户端模块 (conSer 和 conCli)

**conCli** (类) sock, fts\_sock, m\_addr, msg (msgInfo), current\_userid, seraddr, fts\_addr

**conSer** (类) listenfd, port, addr, sql (list<string>), msg (msgInfo), seraddr, peeraddr

## 5.2 登录注册前后端模块设计

由于客户端与服务器的设计过于耦合，所以在这里我们将两端同步介绍。(tips: 此处提及的函数包括：登录注册选择函数：do\_work，服务选择函数：function，前后端消息发送函数：sendmsg。由于机器数量限制，所以我们的登录注册服务器 SER，文件传输服务器 FTS、实时聊天服务器 CHAT 的 ip 地址均一致，但端口相异，这里我们开启各自的独立进程模拟部署在不同机器上的效果)。

```
conCli cli(CPT, "192.168.244.133");
cli.Run();
```

图 5: 指定 ip 地址与连接端口

首先，客户端与服务器会先建立连接，该连接是基于 socket 连接的，系统在初始化时，会先指定好连接的远程服务器的 ip 地址和目标端口，如图5，这个 IP 字符串被存到 conCli 的成员 m\_addr，并被写入 seraddr.sin\_addr.s\_addr (sockaddr\_in seraddr) 用作目标地址；套接字描述符保存在 sock。客户端在程序启动后、调用 conCli::Run() 时执行实际的 TCP 连接：connect(sock, (struct sockaddr \*)&seraddr, sizeof(seraddr)) (在 Run() 的开始处)，连接后进入主循环使用该连接发送/接收管理（登录/注册）消息。如图10所示。

其次，我们的前后端的请求连接的桥梁是基于消息传送的，在这里，我们定义了一个 sendmsg 函数，该函数中调用 socket API send 告诉服务器，我发送的请求类别是什么、用户信息是什么，登陆与注册的消息结构大致相似，主要都是有用户名与密码组成，但是它们的请求类别则分别是 1 和 2，当服务器接收到前端发来的请求后会先识别请求的类别，如果是 1，则该操作是登录操作，那服务器就会查询 mysql，检验该用户是否存在、用户名与密码是否匹配；如果是 2，则该操作是注册操作，那服务器就会先检验该用户是否已经注册，若尚未注册，则将其用户名与密码写入 mysql 中。

```
if (strncmp("1", msg.c_str(), 1) == 0 ||  
    strncmp("register success!", msg.c_str(), 10) == 0)  
{  
    function();  
}
```

图 6: 登录注册校验代码

具体的校验逻辑过程是这样的：系统在调用 send API 将登录/注册信息传给后端后，后端进行信息校验，然后返回给前端校验结果信息，前端此时调用 recv API 接收该结果信息并且做出判断：注册成功了吗？登录信息对不对？登录的用户是不是已经存在了？如果注册成功、登录成功了，那么前端就会调用 function 函数，让用户进入到接下来的服务选择面板，如果不成功，则会调用 do\_work 函数，也就是返回到登录注册界面。校验代码如图6所示。

那么服务器的设计是什么样的？依据前文所述，该服务器会监听 6666 端口，在接收到请求消息后并检验完是请求的操作类型，如果是登录，先尝试 MCache.getValue(userid) (memcached)；若命中且匹配 MD5 则返回成功；否则查询 MySQL (select password from user where user\_id='...')，若与客户端 MD5 相同则缓存并返回成功。如果是注册检查 exists(...)，若不存在则 insert into user(user\_id,password) 并缓存 MD5。登录成功返回"1"，失败返回错误信息字符串。整体的时序图如图7所示。

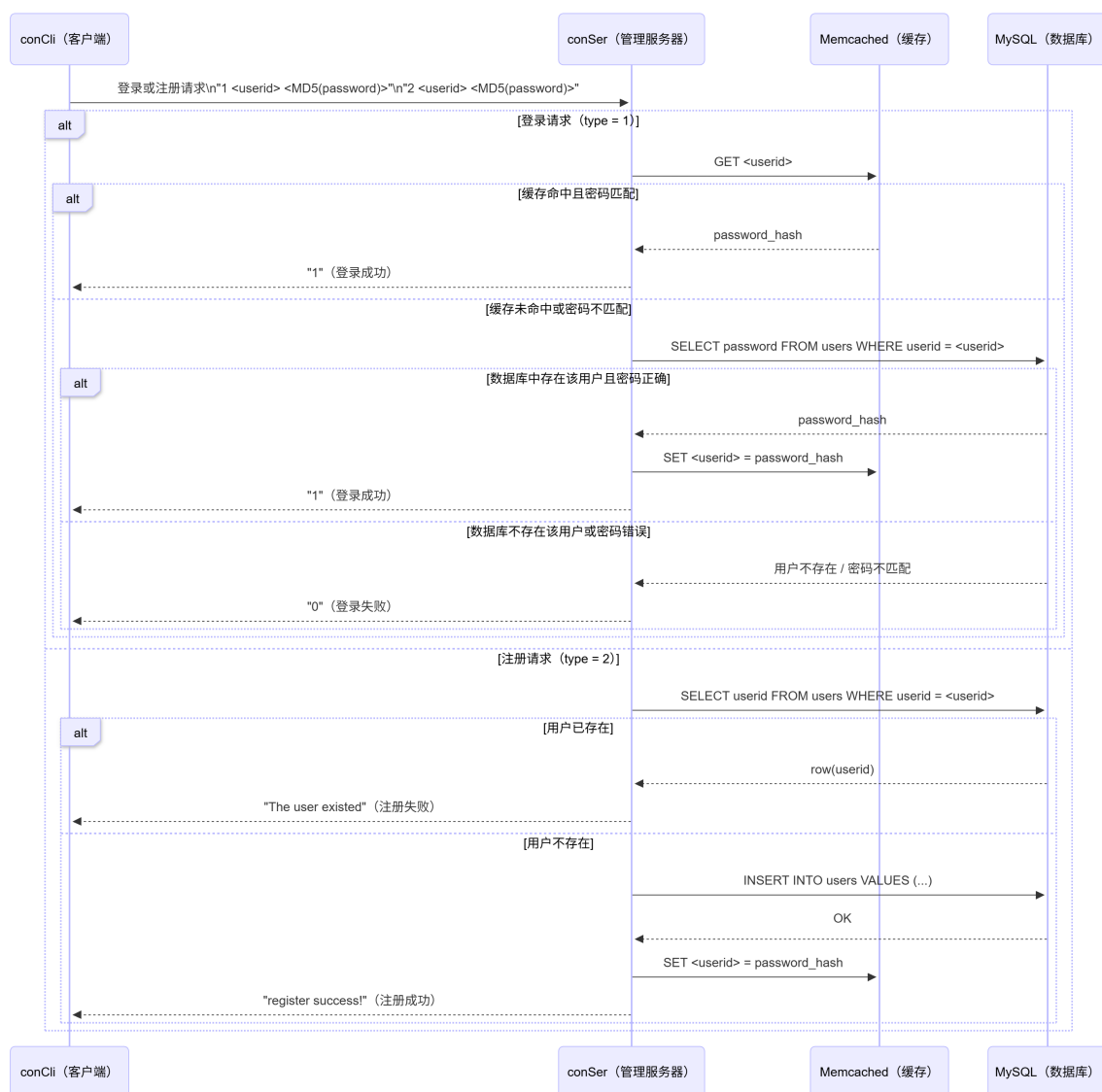


图 7: 登录注册时序图

### 5.3 文件传输前后端模块设计

首先我们会开启 FTS（独立的文件传输服务）来监听 6667 端口。在前端，经过登录/注册，我们顺利进入服务选择界面，如图13，可以关于文件传输的选项一共有三个：1. 上传文件、2. 下载文件、4. 查看文件。每一个选项中都会执行一次独立的与 FTS 服务器连接的 connectFTS 的操作，connectFTS 时 conCli 的成员函数，connectFTS 函数中会有常规的创建套接字 `fts_sock`，并且调用 connect API 与 FTS 服务器进行连接的操作。接下来我会详细介绍这三个功能中的具体操作。

在上传文件这个操作中，前端调用 `uploadFile` 这个接口，而后系统会先让我们输入我们想上传的文件的本地路径，然后系统解析该路径，将其转换成 WSL 路径，例如：`F://submission.txt -> /mnt/f/submission.txt`。接下来系统会去本地磁盘寻找该文件是否存在，如果存在，则计算其 MD5，并尝试连接 FTS 服务器（调用 `connectFTS` 函数），

连接成功后，构建上传命令：3 userid file\_name file\_md5 file\_path file\_size ... 这里的上传文件命令的操作标识是 3，通过 send API 发送请求到 FTS 服务器，服务器处解析完请求命令后发现是 3 操作——上传操作，那么就会返回确认信息给客户端，客户端后续接收到确认后，则将文件内容分块发送给 FTS 服务器，服务器在后端首先会调用 getFileInfo 函数解析传输文件的信息，然后调用 recvFile，在该函数中，系统会校验解析完的 MD5，在 memcached 中去查询这个 MD5，如果能查到该值，则跳过 createFile，实现秒传，如果没有，则调用 createFile，在服务器本地磁盘为每个用户建立独立的存储空间，然后创建文件，将文件内容分块写入到创建的文件中，并且将文件记录写入进 mysql 中，mysql 中的记录信息：file\_name,user\_id,file\_MD5,file\_path,file\_size,status。

在查看文件的操作中，前端调用 listFiles 接口，过程与上传文件类似，先与 FTS 服务器建立连接，而后构建请求命令：7 userid，服务器接收请求命令并解析，发现是查看文件的操作，而后根据 userid 去查询 mysql 中文件记录 (id|file\_name|file\_size)，查询后会将查询到的文件列表返回给前端，前端在 recv 到文件列表信息后会将其呈现给用户。

在下载文件的操作中，前端调用下载 downloadFile 接口，首先会调用之前的查看文件操作，将该用户的所有存储文件显示在界面中供用户下载使用，而后用户可以选择自己想下载的文件名字，系统的 connectFTS 操作与之前的类似，构建请求命令：4 userid filename。服务器解析请求类型为 4，表示是下载文件操作，所以系统调用 sendFile 函数查询数据库获取文件路径/大小，而后在服务器本地磁盘读取该文件，最后使用 sendfile API 分块发送文件内容。客户端在接收到文件内容后，会在客户端的下载路径（预先指定好的）创建文件，然后将接收的文件内容分块地写入该文件。

本模块的设计在客户端与服务器端均存在明显不足，客户端方面主要包括：仅发送本地计算的 MD5 而不验证服务端校验结果，认证依赖明文凭证或硬编码地址，缺少基于 token 的鉴权与 TLS 加密；路径处理与本地/远端路径转换缺乏严格校验，可能导致路径穿越；上传过程为一次性流式写入，没有断点续传、分块校验、失败重试或临时文件（.part）与原子重命名机制，网络中断后无法恢复；发送与接收均为阻塞 I/O，界面逻辑与网络操作互相阻塞，缺少异步化、非阻塞策略与进度回调；客户端对管理端或 memcached 返回的信息（如秒传结果、FTS 地址）缺乏二次确认。服务器端方面同样存在：FTS 对客户端提供的 MD5 信任过高，memcached 命中时跳过实际校验，且没有在缓存命中后到磁盘或数据库确认文件存在性的回退机制，导致潜在伪造与数据不一致；服务端虽计算 server-side MD5，但仅记录日志，并不作为强制校验或回滚依据；FileTrans 采用 fork-per-connection 与阻塞磁盘写策略，高并发或大文件场景下容易资源耗尽；用户请求可信度过高，缺少路径合法性检查、目录限制与原子重命名，部分写入可能留下损坏文件而数据库记录不完整；通信不加密，无 TLS；缺乏资源配额、磁盘可用性检测、失败回滚、对缓存陈旧性的处理，以及对并发访问（如 connfd、shared 数据结构、DB 连接池）的保护不足，整体导致数据完整性、可用性与安全性存在风险。

文件传输的时序图如图8所示。

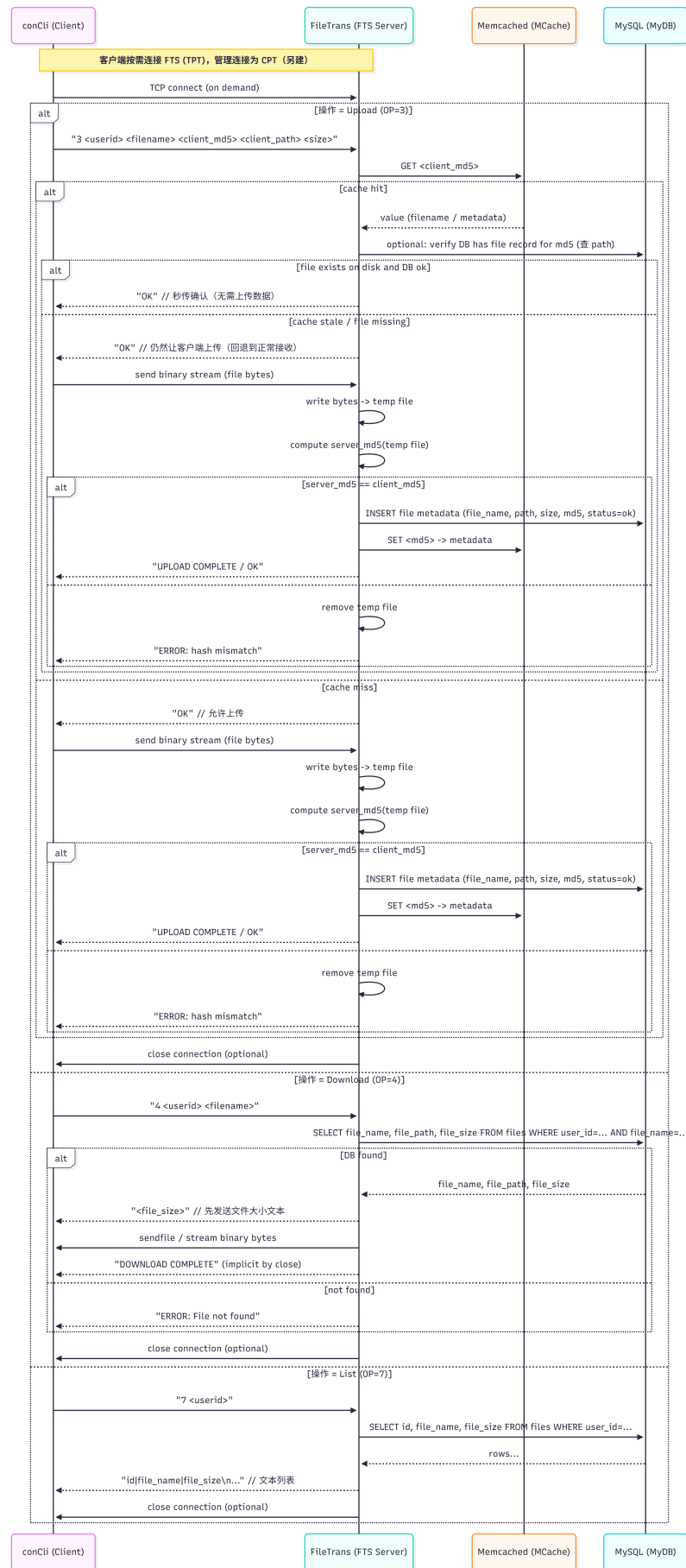


图 8: 文件传输时序图



## 5.4 实时聊天前后端模块设计

首先我们会开启一个独立的 CHAT 进程，CHAT 进程监听的端口是 10808。在客户端，聊天时会先创建 `chat::ChatClient` 实例并初始化日志，调用 `connect(uri)` 建立到聊天服务器的 WebSocket 连接（例如 `ws://<ip>:10808`），在连接建立后注册消息回调用于接收异步推送并在回调里用 `client/common/message` 的解析函数把文本帧解析为 `Message` 对象并在本地显示（通常只显示非自己发送的消息），在主循环里读取用户输入并调用 `client.send(text)` 发送文本帧到服务器，同时处理连接失败、超时或断开时的提示或重连逻辑，并在退出时调用 `disconnect()` 清理资源；客户端还负责把发送的消息做本地格式化（当前为自定义字符串格式）、记录日志、以及在需要时与管理/FTS 模块交互以完成鉴权或获取其他服务地址。

聊天服务器启动时初始化 `websocketpp / ASIO` 监听指定端口并在 `on_open` 将新连接加入到连接集合以维护活动会话，在 `on_message` 回调接收到文本帧后解析为 `Message`（当前为自定义文本格式）并将消息广播到除发送者外的连接集合，同时调用预设的 `messageCallback` 将消息异步或同步持久化到存储层（例如数据库或日志），在 `on_close` 从连接集合移除断开的客户端并清理资源，整个服务器负责连接管理、消息分发、日志记录和可选的持久化，但当前实现对鉴权、心跳和连接集合并发访问保护较弱（建议在 `on_open` 验证 token、实现 ping/pong、并为 `connections` 加锁或使用 `asio::strand` 以保证线程安全和稳定性）。

聊天客户端目前还有一些缺陷，例如，没有在连接阶段或首条消息中强制进行可靠鉴权，通常只用用户名标识，易被伪造；消息采用自定义文本格式并以分隔符解析，脆弱且不利于扩展，缺少统一的 JSON 或结构化协议；缺少消息 ID、ACK 和重试机制，无法保证投递语义或去重；没有可靠的自动重连/指数退避策略与断线恢复流程，UI 输入（如 `getch`）可能阻塞或与网络回调竞争；对消息大小、内容和速率缺乏限制和校验，易受滥发或注入攻击；未明确实现心跳/ping-pong 以探测死连接；日志/错误处理与回退机制不完善，离线消息和本地缓存支持缺失；传输层通常为明文（未使用 `wss/TLS`），敏感信息与消息内容可能被窃听。服务器设计也有些许缺陷，例如连接集合（`connections`）的并发访问未受保护或没有使用 `asio::strand`，在多线程环境容易发生数据竞争和迭代器失效；在 `on_open/on_message` 未强制验证 token 导致未授权连接被接纳；消息格式脆弱且无版本控制，广播为 `fire-and-forget`，缺少消息 ID、客户端 ACK、持久化确认与重试机制，导致无法保证消息可靠投递或顺序；持久化逻辑可能在 IO 回调线程中同步执行，阻塞事件循环并降低吞吐；缺乏心跳/超时剔除、速率限制与连接数配额，易受 DOS/滥用；广播缺少背压处理与分片策略，当大量连接或大消息时会导致资源耗尽；缺乏房间/权限模型和细粒度授权，所有消息默认广播且无法有效隔离；安全性不足（未强制 `TLS`、不校验消息来源、不限制消息体大小或内容），并且没有跨实例扩展方案（如 `Redis Pub/Sub` 或消息队列），因此水平扩展困难且无法保证跨进程的一致性与可用性。

实时聊天时序图如图9所示

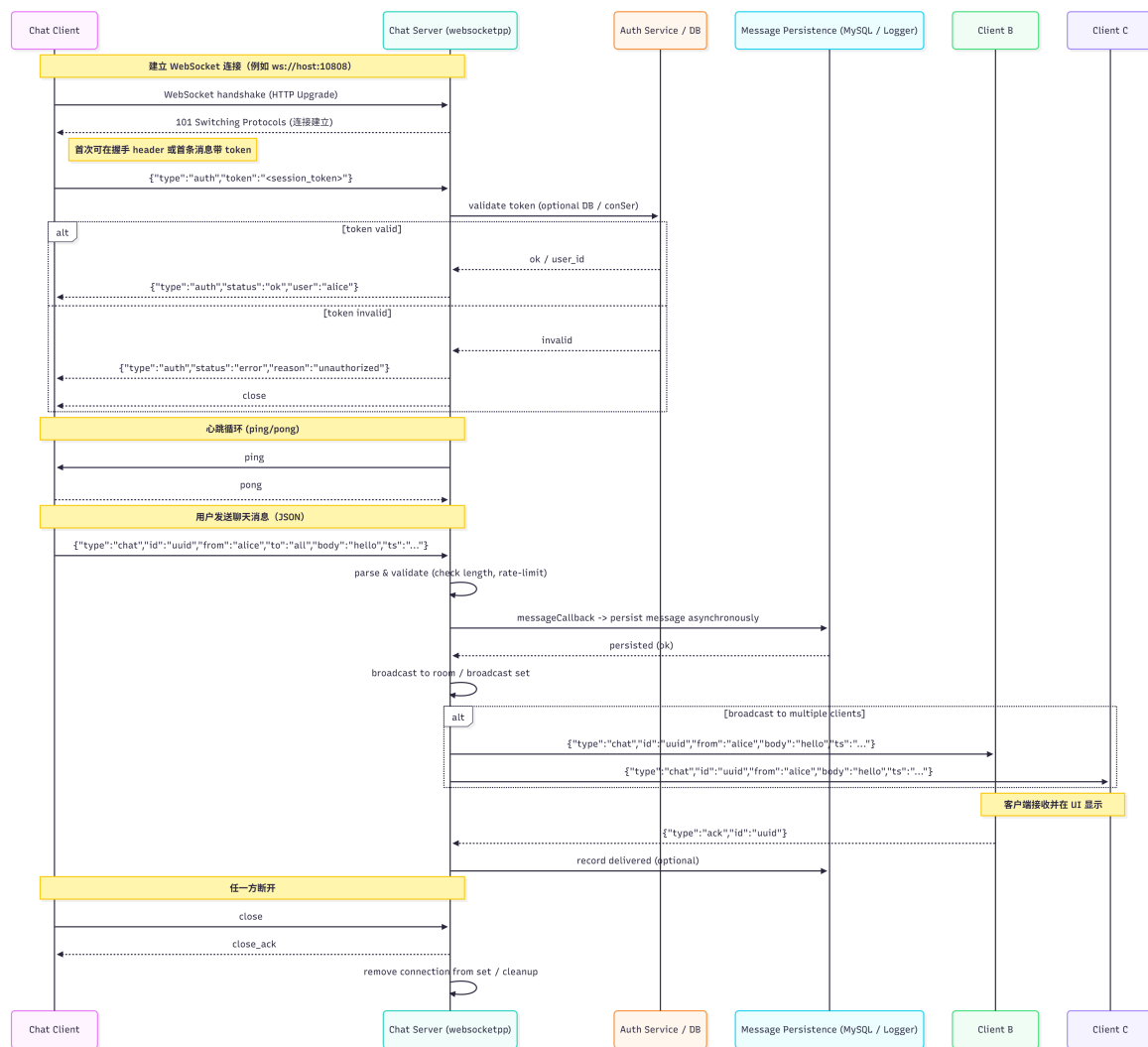


图 9: 聊天时序图

## 6 功能展示与分析

本章节我将分别展示本项目的核心功能并对各个功能进行具体的分析。用于测试的环境：一台 Ubuntu-24.04.3 的虚拟机（ip 地址为 192.168.244.133），一台 windows11 主机，使用的 wsl 版本为 2.6.1.0。

### 6.1 登录注册功能分析

在顺利准备好所有运行的环境后，在客户端启动 runClient 脚本，成功运行后我们会进入一个登录注册的交互界面，选择选项 2 后，根据提示输入用户名和密码，如图10所示。成功注册后，数据库中将会写入该用户的所有信息，这时我们检查数据库会发现，该 aTestAccount 用户已经被写进数据库，并且为了安全起见，密码也经过 MD5 加密处理，并不是，输入的 123321。查询数据库情况如图11所示。

```
=====
= Welcome To My Network Disk! =
=====
=                               =
= 1.登录      2.注册            =
=                               =
=====
Your Choice >: 2
用户名: aTestAccount
密 码: 123321
```

图 10: 注册界面

```
mysql> select * from user;
+-----+-----+-----+
| user_id | password | created_at |
+-----+-----+-----+
| aTestAccount | c8837b23ff8aaa8a2dde915473ce0991 | 2025-11-19 17:33:45 |
| cory | 76f5d947149185d77a1fa1a114b3fb30 | 2025-11-16 23:35:12 |
| jon | c8837b23ff8aaa8a2dde915473ce0991 | 2025-11-16 23:35:35 |
| mary | c8837b23ff8aaa8a2dde915473ce0991 | 2025-11-16 23:39:39 |
| wang | c8837b23ff8aaa8a2dde915473ce0991 | 2025-11-16 23:34:03 |
| wangzhihao | c8837b23ff8aaa8a2dde915473ce0991 | 2025-11-16 23:47:15 |
+-----+-----+-----+
```

图 11: 检查用户数据库

在注册成功后即可进行登录操作，用户可以输入选项 1，并且输入自己的用户名和密码，即可完成登录，并且跳转到功能选项窗口界面，如图12和图13所示。

```
=====
= Welcome To My Network Disk! =
=====
=                               =
= 1.登录      2.注册            =
=                               =
=====
Your Choice >: 1
用户名: wang
密 码: 123321
```

图 12: 登录界面

```
=====
= Welcome To My Network Disk! =
=====
=                               =
= 1.上传文件      2.下载文件      =
= 3.聊天          4.查看文件      =
= 0.返回登录      =
=                               =
Your Choice >: |
```

图 13: 成功登入

## 6.2 文件传输功能分析

本项目的文件传输功能主要分为三个板块：1. 上传文件，2. 下载文件，3. 查看已存储的文件。三种功能完美符合云盘系统所需要的基本功能。接下来将一一演示。

### 6.2.1 文件上传

```
=====
= Welcome To My Network Disk! =
=====
=                               =
= 1.上传文件      2.下载文件      =
= 3.聊天          4.查看文件      =
= 0.返回登录      =
=                               =
Your Choice >: 1

===== 文件上传 =====
[INFO] 如果使用 Windows 路径 (如 F://file.txt) , 会自动转换为 WSL 路径 (/mnt/f/file.txt)
请输入要上传的文件路径: F://wang-testfile.txt
```

图 14: 上传界面

首先是上传功能，在选择功能界面，我们输入选项 1，系统会提示我们需要输入你想上传的文件的路径，如图14所示并且如果你使用的是 Windows 系统的 WSL 服务，无

需担心因为不熟悉 Linux 系统而不知道怎么输入相应的 WSL 路径，直接输入 Windows 系统磁盘的真实路径即可，因为系统会自动转换成相应的 WSL 路径，这极大地便利了对 Linux 不熟悉的一般用户。输入完成后系统就会发送文件到服务器进行存储，如图15所示，传输成功。

```
=====
=   Welcome To My Network Disk!   =
=====
=
=   1.上传文件      2.下载文件      =
=   3.聊天         4.查看文件      =
=   0.返回登录     =
=
=====
Your Choice >: 1

===== 文件上传 =====
[INFO] 如果使用 Windows 路径 (如 F://file.txt) , 会自动转换为 WSL 路径 (/mnt/f/file.txt)
请输入要上传的文件路径: F://wang-testfile.txt
[INFO] 转换后的路径: /mnt/f/wang-testfile.txt
[INFO] 文件大小: 31 字节
[INFO] 文件 MD5: 837507b08e9987c13d557c844ba5789a
[INFO] 连接到 FTS 服务器: 192.168.244.133:6667
[DEBUG] 正在连接 FTS 服务器: 192.168.244.133:6667
[INFO] 已成功连接到 FTS 服务器
[DEBUG] 上传命令: 3 wang wang-testfile.txt 837507b08e9987c13d557c844ba5789a /mnt/f/wang-testfile.txt 31
[DEBUG] 服务器确认: OK
[INFO] 开始上传文件...
[SUCCESS] 文件上传完成!
```

图 15: 成功上传

现在我们检查服务器数据库与本地磁盘中是否存在该文件，首先查询数据库中的所有文件，如图16所示，我们看到了文件 wang-testfile.txt 的信息和索引已经被导入进数据库。

| id | file_name         | user_id    | file_MD5                         | file_path                           | file_size | status | created_at          |
|----|-------------------|------------|----------------------------------|-------------------------------------|-----------|--------|---------------------|
| 1  | submission.txt    | wangzhihao | b723cb73bfc0f19c4933c2ebbf8341b  | ./storage/wangzhihao/submission.txt | 897624    | ok     | 2025-11-17 01:15:45 |
| 2  | submission.txt    | wang       | b723cb73bfc0f19c4933c2ebbf8341b  | ./storage/wang/submission.txt       | 897624    | ok     | 2025-11-17 14:18:29 |
| 3  | wang-test.txt     | wangzhihao | e7b1e2f5e4e2c96e18b141805a8aec87 | ./storage/wangzhihao/wang-test.txt  | 31        | ok     | 2025-11-19 17:24:39 |
| 6  | wang-testfile.txt | wang       | 837507b08e9987c13d557c844ba5789a | ./storage/wang/wang-testfile.txt    | 31        | ok     | 2025-11-20 23:36:26 |
| 7  | wang-testfile.txt | wang       | 837507b08e9987c13d557c844ba5789a | /mnt/f/wang-testfile.txt            | 31        | error  | 2025-11-20 23:37:00 |
| 8  | wang-testfile.txt | wang       | 837507b08e9987c13d557c844ba5789a | /mnt/f/wang-testfile.txt            | 31        | error  | 2025-11-20 23:37:20 |

6 rows in set (0.00 sec)

图 16: 检查数据库

而后我们检查相应的实际存储磁盘路径，可以从图17中看出，文件已经被写入磁盘中，并且每个用户都独立地拥有一个属于自己的磁盘存储路径，这样互相互相不干扰，很好地实现了信息的隔离以及隐私的保护。

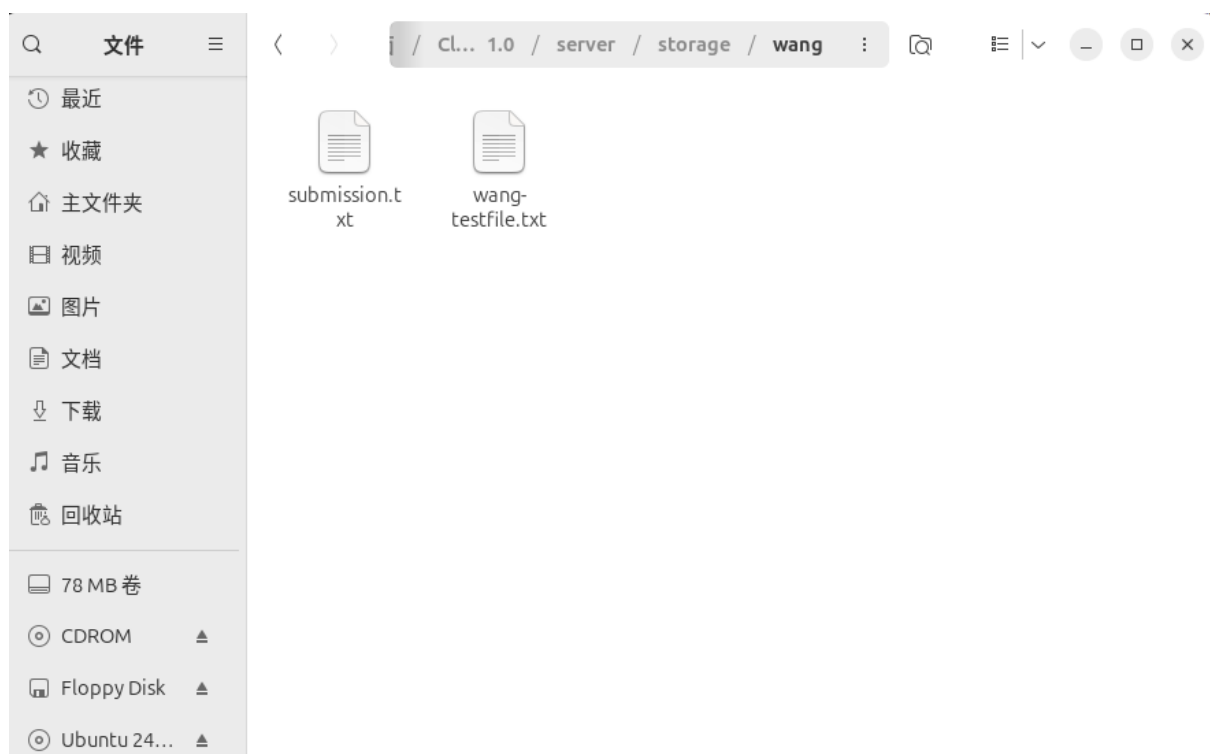


图 17: 检查磁盘

### 6.2.2 文件下载

文件在上传存储后，用户如果想从云盘中下载自己想要的文件也是非常简便的，只需要在交互面板上输入选项 2，并等待系统加载用户已存入云盘中的文件列表，如图18所示。从中我们可以得到文件的文件名和文件大小的信息。

```
===== 文件列表 =====
[INFO] 连接到 FTS 服务器: 192.168.244.133:6667
[DEBUG] 正在连接 FTS 服务器: 192.168.244.133:6667
[INFO] 已成功连接到 FTS 服务器
[INFO] 你的文件列表:
8|wang-testfile.txt|31
7|wang-testfile.txt|31
6|wang-testfile.txt|31
2|submission.txt|897624

请输入要下载的文件名: wang-testfile.txt
[INFO] 连接到 FTS 服务器: 192.168.244.133:6667
[DEBUG] 正在连接 FTS 服务器: 192.168.244.133:6667
[INFO] 已成功连接到 FTS 服务器
[DEBUG] 下载请求: 4 wang wang-testfile.txt
[INFO] 文件大小: 31 字节
[INFO] 开始接收文件...
[SUCCESS] 文件下载完成! 保存到: ./storage/wang-testfile.txt
```

图 18: 下载文件

接下来我们验证文件是否已经下载本地，本项目的存储路径默认是./client/storage/，其中 storage 无需用户提前创建，系统在传输文件时会自动创建，如图19，我们可以看到文件 wang-testfile.txt 已经成功下载到本地。

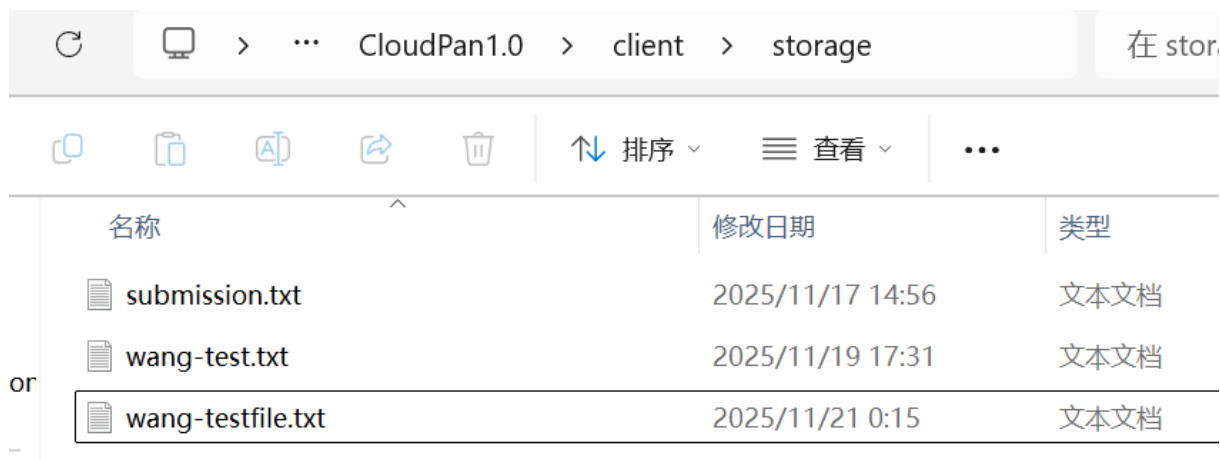


图 19: 验证是否下载成功

### 6.2.3 查看文件

查看文件的操作其实在下载文件的时候已经内嵌，但是当用户没有下载需求只是想要查看云盘存储内容时，单独的查看文件的功能就必不可少。当我们在交互界面输入选项 4 后，系统则会在面板中显示当前用户在云盘中存储的所有文件的信息，如图??所示。

```
=====
=  Welcome To My Network Disk!  =
=====
=                                =
=  1.上传文件      2.下载文件    =
=  3.聊天          4.查看文件    =
=  0.返回登录      =
=                                =
Your Choice >: 4

===== 文件列表 =====
[INFO] 连接到 FTS 服务器: 192.168.244.133:6667
[DEBUG] 正在连接 FTS 服务器: 192.168.244.133:6667
[INFO] 已成功连接到 FTS 服务器
[INFO] 你的文件列表:
8|wang-testfile.txt|31
7|wang-testfile.txt|31
6|wang-testfile.txt|31
2|submission.txt|897624
```

图 20: 查看文件

## 6.3 实时聊天功能分析

本项目中一大亮点就是在云盘系统的基础上嵌入了实时聊天的模块，系统会提供一个默认的大聊天室，所有该系统的会员都可以在此聊天室中进行交流，开启的方式同上

述各功能相似，只需在交互界面中输入选项 3 即可进入公共聊天室，例如，我现在有三位用户已经进入了该聊天室中，他们分别是 wang、wangzhihao、mary，每一位用户都向聊天室中发送一句问候，我们发现可以在每一位的聊天框中接收到所有人发送的问候消息，如图21所示。这样多用户聊天的功能我们也成功实现。

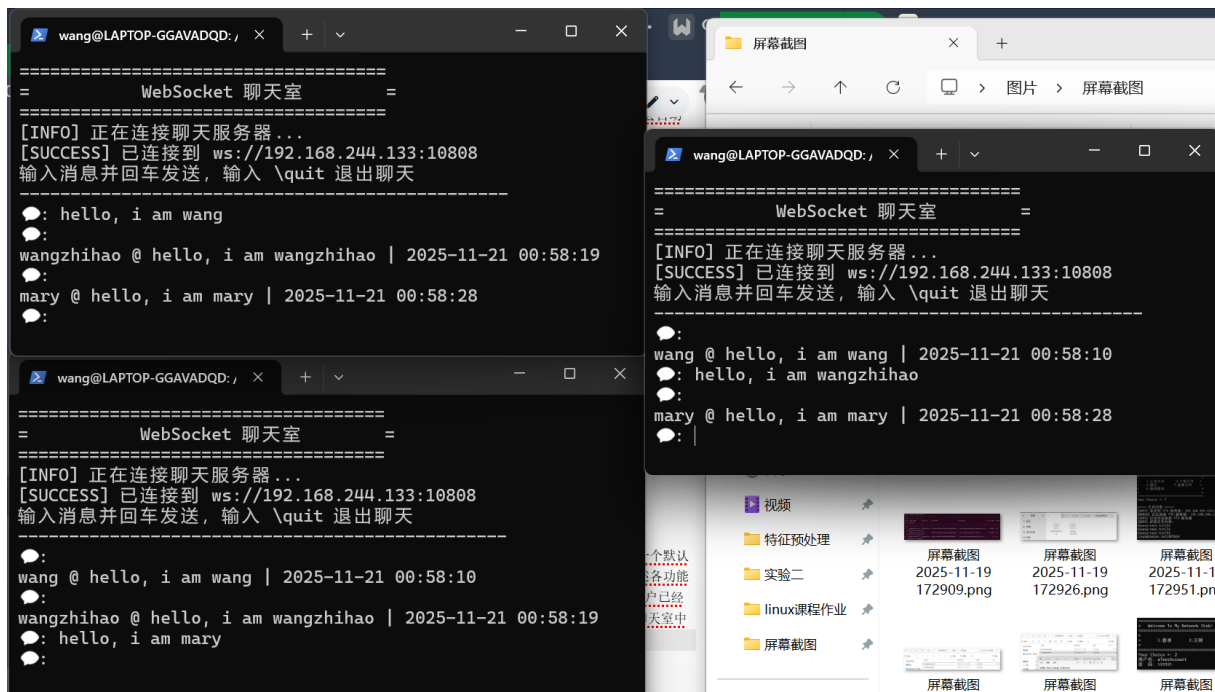


图 21: 聊天室

现在我们查看后端服务器中的监听情况以及是否记录了三位用户的聊天记录，如图22和图23所示，服务器实时监听着三位用户的聊天信息，并且聊天记录也正常写入了服务器的 chat-history 日志中。



图 22: 后端监听聊天

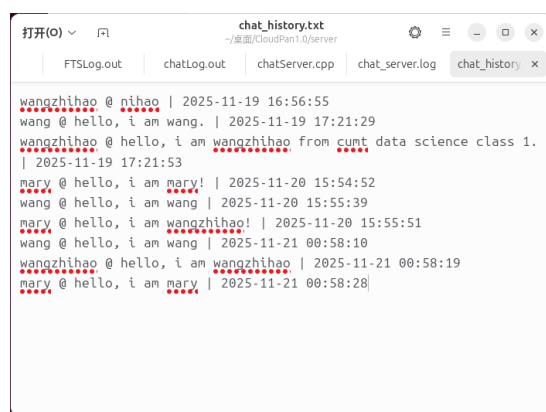


图 23: 聊天记录

## 7 开发环境与项目结构

- 操作系统版本: Windows11, Ubuntu24.04.3

- 编译器版本: gcc (tdm64-1) 4.9.2
- 使用的库: websocketpp, Asio / Boost.Asio, libmysqlclient (MySQL C API) , libmemcached, POSIX Socket API (Linux 系统调用) , pthread (POSIX 线程库) , libstdc++ / C++ 标准库, Linux 系统工具库.
- 工程目录结构:

```
CloudPan1.0/
├── .vscode/
├── client/
│   ├── chat_client/
│   │   ├── main.cpp
│   │   ├── websocket_client.cpp
│   │   └── websocket_client.hpp
│   ├── common/
│   │   ├── logger.cpp
│   │   ├── logger.hpp
│   │   ├── message.cpp
│   │   └── message.hpp
│   ├── conCli.cpp
│   ├── conCli.h
│   ├── login.h
│   ├── logs/
│   ├── Makefile
│   ├── MCache.h
│   ├── md5.cpp
│   ├── md5.h
│   └── storage/
├── server/
│   ├── chat_server/
│   │   ├── websocket_server.cpp
│   │   ├── websocket_server.hpp
│   │   └── main.cpp
│   ├── conSer.cpp
│   ├── conSer.h
│   ├── fileTrans.cpp
│   ├── fileTrans.h
│   └── Makefile
```



```

|   |— md5.cpp
|   |— md5.h
|   |— MyDB.cpp
|   |— MyDB.h
|   |— ser
|   |— serLog.out
|   |— sqlConnPool.cpp
|   |— sqlConnPool.h
|   |— tPool.cpp
|   |— tPool.h
|— Makefile
|— public.h

```

## 8 使用说明

### 8.1 编译方式

安装编译与运行所需库（以 Ubuntu 为例）：

```

1 sudo apt update
2 sudo apt install build-essential g++ libmysqlclient-dev libmemcached-
  dev libssl-dev libboost-system-dev libboost-thread-dev memcached
  mysql-server

```

分别构建 Client / Server：

```

1 g++ conCli.cpp md5.cpp chat_client/websocket_client.cpp common/logger.
  .cpp common/message.cpp -o cli -I. -lpthread -
  lboost_system -lboost_random -std=c++11
2 g++ -o ser conSer.cpp MyDB.cpp -I /usr/include/libmemcached/include -
  L/usr/lib/mysql -lmysqlclient -lmemcached -lpthread --std=gnu++0x
3 g++ -o FTS fileTrans.cpp md5.cpp MyDB.cpp -I /usr/include/
  libmemcached/include -L/usr/lib/mysql -lmysqlclient -lmemcached -
  lpthread --std=gnu++0x
4 g++ ./server/main.cpp ./server/websocket_server.cpp ./common/logger.
  cpp ./common/message.cpp -o chat_server -lpthread -lboost_system -
  std=c++11

```

### 8.2 数据库与缓存准备

```
1 sudo service mysql start
2 sudo service memcached start
3
4 -- 登录 mysql 后执行示例
5 CREATE DATABASE CloudPan DEFAULT CHARACTER SET utf8mb4;
6 USE CloudPan;
7 CREATE TABLE users (
8     id INT AUTO_INCREMENT PRIMARY KEY,
9     userid VARCHAR(64) UNIQUE,
10    password VARCHAR(128)
11 );
12 CREATE TABLE files (
13     id INT AUTO_INCREMENT PRIMARY KEY,
14     file_name VARCHAR(255),
15     user_id VARCHAR(64),
16     file_MD5 VARCHAR(128),
17     file_path VARCHAR(1024),
18     file_size BIGINT,
19     status VARCHAR(16)
20 );
```

### 8.3 运行方式

在后台分别启动 FTS, ser, chat:

```
1 # 在 server 目录（或项目根）运行
2 ./FTS
3 # 若在后台
4 nohup ./FTS > logs/fts.log 2>&1 &
5
6 ./ser
7 # 或后台
8 nohup ./ser > logs/ser.log 2>&1 &
9
10 ./chat
11 # 或后台
12 nohup ./chat > logs/chat.log 2>&1 &
```

在客户端启动 cli:

```
1 在另一终端或多台机器上  
2 ./cli
```

## 9 总结

本课程设计实现了一个简易的云盘系统，包含管理服务器（用于用户注册/登录）、文件传输服务器（FTS，用于上传/下载/列出文件）、客户端命令行界面（支持登录、上传、下载、查看文件、WebSocket 聊天）以及一个基于 websocketpp 的聊天子系统。项目覆盖了网络编程（TCP/Socket、epoll、sendfile）、数据库交互（MySQL C API）、缓存（memcached）、并发（fork/线程池设计）和异步消息（WebSocket + ASIO）等技术栈，具有完整的端到端交互流程和可运行的原型。

通过实现本项目掌握了套接字编程的端到端流程，包括 socket 创建、bind/listen/accept、connect、send/recv 的使用；学会使用 epoll 和 fork 简化并发连接处理，对 I/O 多路复用的基本思想有了直观理解；实践了 sendfile() 等零拷贝优化、了解文件分块与接收循环的实现要点；理解了数据库访问流程（如何用 C API 执行查询、解析结果并映射到业务逻辑），并通过 memcached 实现简单的缓存加速和秒传思路；掌握了 websocketpp 与 Asio 的基本用法，体验了实时消息推送的设计；同时通过实现客户端交互、路径转换、MD5 校验等功能，提升了工程化思维（模块划分、日志、简单错误处理与用户交互设计）。

本次项目加深了我对网络协议与传输细节的理解（控制消息与二进制流的分离、何时发送元信息、如何传递文件大小以便接收端预分配/读循环）；提升了 C/C++ 系统编程能力（文件 I/O、权限、目录处理、文件描述符管理、进程/线程模型）；学习了如何把缓存（memcached）与数据库（MySQL）结合到应用逻辑中以提升性能；通过 WebSocket 实现实时通信，理解了消息序列化、事件回调与异步处理模式；增强了调试与诊断能力（在代码中增添日志、调试输出、边界条件处理等）。

当前实现中存在若干设计与安全欠缺：认证与授权不够完善（客户端使用 MD5 明文发送 + 服务端信任 userid 文本，未使用 session token/JWT）；传输过程中缺少端到端完整性保障（服务端虽然计算了 MD5 但并未强制比对与回滚），memcached 的秒传逻辑信任客户端提供的 MD5，可能被伪造；SQL 语句仍以拼接为主，存在注入风险，未广泛使用预处理语句与事务；并发处理方面部分组件（如聊天服务的连接集合）缺乏明确的线程同步策略，FileTrans 使用 fork-per-connection 在高并发下可扩展性与资源效率有限；文件传输缺少分块/断点续传、传输速率控制和临时写入（.part）+ 原子重命名流程，发生中断时易留下残损文件或不一致的 DB 记录；整个系统默认明文传输（无 TLS），缺乏速率限制、权限校验、日志审计与入侵防护等生产级要素。

短期可做的改进包括：在文件接收后强制计算并比对 server-side hash 与 client-side hash，只有一致时才写入 DB 与 memcached；把客户端与服务器之间的地址/端口改为配置驱动，避免硬编码；将 SQL 改用预处理语句并对关键写操作加事务保护；在聊天

模块对连接集合加互斥或使用 `asio::strand`，并引入 JSON 消息格式与鉴权 token；中期可加入分块上传/断点续传、上传临时文件 + 校验后重命名、`sendfile` + 限速策略与连接池化设计以提高并发性能；长期可考虑使用 TLS (`wss/HTTPS`)、引入消息队列/Redis PubSub 实现聊天多实例扩展、将文件存储迁移到对象存储 (S3/NAS) 并实现更完善的审计与监控。

## 10 附录

由于代码工程量过大，因此具体代码不直接在报告中呈现，具体详细的代码以及对应的注释请见附带的源代码包。这里展示重要文件中的重要函数与注释：

`conCli.cpp`:

```
1 #include "md5.h"
2 #include "conCli.h"
3 #include <sstream>
4 #include <cstdio>
5 #include <cctype>
6 #include "common/logger.hpp"
7 #include "chat_client/websocket_client.hpp"
8 #include <termios.h>
9 #include <unistd.h>
10
11 /*
12  * getch
13  * 非回显单字符读取（类似于 conio.h 的 getch），用于在终端环境下逐字符读取输入以
14  * 支持即时回显与退格处理。该函数会临时关闭终端行缓冲与回显，然后读取一个字符，
15  * 读取完成后恢复终端设置并返回字符。调用者需确保在交互循环中合理使用。
16  */
17 char getch()
18
19 /*
20  * conCli 构造函数
21  * 参数：port - 目标管理服务器端口（CPT），addr - 管理服务器 IP 字符串
22  * 行为：创建 TCP socket，并初始化目标地址结构 `seraddr`（但不进行 connect 操作），
```

```

23  *      将传入的地址保存到成员 `m_addr` 以备后续在 Run() 中 connect
      使用。
24  */
25  conCli::conCli(int port, const char *addr)
26
27  /* 析构函数：目前不做特殊清理 (socket 在 Run 流程中关闭)。 */
28  conCli::~conCli() {}
29
30  /*
31  * Run
32  * 行为：对管理服务器发起一次 connect 并进入交互循环。该连接用于登录/
      注册及后续
33  *      管理命令的同步 send/recv。函数会阻塞直到连接建立或发生致命错
      误。
34  */
35  void conCli::Run()
36
37  /* 打印欢迎界面 (登录 / 注册 选择) */
38  void conCli::wellcome()
39
40  /*
41  * function
42  * 主功能菜单：处理上传/下载/聊天/查看文件等操作的用户选择并调用对应
      函数。
43  * 该接口为阻塞式 CLI，函数内部通过循环读取用户选择并分发。
44  */
45  void conCli::function()
46
47  /*
48  * sendmsg
49  * 说明：将 `msg` 字符串发送到管理服务器 (通过成员 socket `sock`)，
      然后阻塞等待一次响应。
50  *      响应保存在 `msg` 中以便调用者进一步解析。注意这是同步阻塞通
      信，适合简单 CLI 场景但
51  *      在 GUI/异步场景中会造成阻塞。错误分支会尝试返回到登录/主菜
      单。
52  */
53  void conCli::sendmsg()
54
55  /*

```

```

56  * getinfor
57  * 从标准输入读取用户名与密码，将密码通过本地 MD5 散列后拼接到 `msg`
    中并调用 sendmsg 发送。
58  * 注意：在真实生产环境中不应在客户端使用 MD5 作为密码传输凭证，且应
    通过 TLS 保护通道并
59  *      使用更安全的认证方式（如一次性口令、token、加盐哈希等）。
60  */
61 void conCli::getinfor()
62
63 /*
64  * do_work
65  * 读取登录/注册选择并调用 getinfor 完成凭证输入与发送。选择由 `
    OP_TYPE` 枚举决定（在 public.h）。
66  */
67 void conCli::do_work()
68
69 /*
70  * connectFTS
71  * 参数：fts_ip, fts_port - 输出参数，将被设置为要连接的 FTS IP 与端
    口。
72  * 行为：按需创建一个新的 socket (`fts_sock`) 并连接到文件传输服务（
    FTS）。当前实现为
73  *      使用硬编码的 IP/端口（可改为从配置或管理服务器获取）。返回
    true 表示连接成功。
74  */
75 bool conCli::connectFTS(string &fts_ip, int &fts_port)
76
77 /*
78  * chatRoom
79  * 处理聊天室交互：创建 ChatClient、连接到 WebSocket 服务器、注册消息
    回调，
80  * 在本地循环读取用户输入并发送消息，支持退出命令。注意本函数为阻塞式
    交互。
81  */
82 void conCli::chatRoom()
83
84 上传文件的前端接口函数：
85 void conCli::uploadFile()
86
87 查看云端文件列表的前端接口函数：

```

```

88 void conCli::listFiles()
89
90 下载文件的前端接口函数：
91 void conCli::downloadFile()
92
93 int main()

```

conSer.cpp:

```

1  #include "MCache.h"
2  #include "conSer.h"
3  #include <sys/wait.h>
4
5  /*
6   * conSer 构造函数
7   * 参数：port - 监听管理命令的端口（CPT），addr - 绑定地址（当前实现
           绑定 INADDR_ANY）
8   * 行为：创建用于管理连接的监听 socket 并初始化地址结构，不在此处
           bind/listen（在 Run 中统一调用）。
9   */
10 conSer::conSer(int port, const char *addr)
11
12 /* 析构函数：当前不做特殊清理（若以后引入成员指针可在此释放） */
13 conSer::~conSer()
14
15 /*
16 * Socket
17 * 为监听 socket 设置常用选项（如 SO_REUSEADDR）。该函数单纯做选项配
           置，错误则退出。
18 */
19 void conSer::Socket()
20
21 /*
22 * Bind
23 * 绑定监听 socket 到指定地址。将 bind 操作从构造函数中分离，便于更清
           晰的生命周期控制。
24 */
25 void conSer::Bind()
26
27 /*
28 * Listen

```

```

29  * 将 socket 切换为监听状态，准备 accept 连接。
30  */
31 void conSer::Listen()
32
33 /*
34  * Run
35  * 启动服务器的完整流程：设置 socket 选项、绑定、监听并进入 epoll 工
    作循环。
36  */
37 void conSer::Run()
38
39 /*
40  * do_service
41  * 处理单个管理连接的会话：循环读取客户端消息、解析并执行命令（通过
    get_cmd），
42  * 最后把生成的响应写回客户端。该函数运行在 fork 出来的子进程中。
43  */
44 void conSer::do_service(int conn)
45
46 /*
47  * selectFTServer
48  * 将文件传输服务器（FTS）的地址与端口追加到 `msg` 中，供客户端在收到
    管理响应后用作后续上传/下载连接。
49  * 当前实现为简单返回常量 `SERADDR` 与 `TPT`，可扩展为负载均衡或动态
    选取逻辑。
50  */
51 void conSer::selectFTServer()
52
53 /*
54  * main
55  * 程序入口：设置 SIGCHLD 行为以避免僵尸进程，创建 `conSer` 实例并运
    行服务循环。
56  */
57 int main()

```

fileTrans.cpp:

```

1  #include "md5.h"
2  #include "MCache.h"
3  #include "MyDB.h"
4  #include "fileTrans.h"

```



```
5 #include <sys/stat.h>
6 #include <sys/types.h>
7
8 /*
9  * sql_escape
10  * 对传入字符串做最小的 SQL 转义，替换单引号与反斜杠，以降低直接拼接
    字符串时的语法问题。
11  * 注意：此函数不是完整的防注入手段，生产环境请使用预处理语句/参数化
    查询。
12  */
13 static string sql_escape(const string &s)
14
15 /*
16  * FileTrans 构造函数
17  * 参数：port - 监听端口（TPT），addr - 绑定地址（目前未使用，绑定
    INADDR_ANY）
18  * 行为：创建监听 socket、设置 SO_REUSEADDR 并绑定/监听。出错时会直接
    退出（ERR_EXIT）。
19  */
20 FileTrans::FileTrans(int port, const char *addr)
21
22 /*
23  * 析构函数：关闭监听与当前连接描述符（如果已打开）。
24  */
25 FileTrans::~FileTrans()
26
27 /*
28  * Run
29  * 主循环：使用 epoll 监听监听套接字 `trans_fd`，当有新连接到达时
    accept 并 fork 出子进程处理。
30  * 每个子进程负责解析客户端命令（以空格分隔），将参数拷贝到成员变量 `
    sql`，并根据操作类型
31  * 分发到 upload/download/list/other 处理函数。设计采用 fork-per-
    connection（便于隔离和简单并发），
32  * 但在高并发下会有进程开销问题，生产环境可考虑线程池或基于 epoll 的
    事件驱动单进程多路复用。
33  */
34 void FileTrans::Run()
35
36 /*
```

```
37  * getCmd
38  * 通用命令解析器（用于那些没有在主分发中单独处理的操作）。
39  * 将 msg 的参数部分逐项放入成员 `sql`，并根据 OP_TYPE 调用相应处理函
    数。
40  */
41 void FileTrans::getCmd()
42
43 /*
44  * getFileInfo
45  * 从成员 `sql` 中按顺序提取文件信息字段并填充到 `m_fileinfo`：
46  * file_name, file_MD5, file_path, file_size, file_chunk_size,
    chunk_size。
47  * 若某些可选字段缺失，使用合理默认值（0 或 false）。
48  */
49 void FileTrans::getFileInfo()
50
51 /*
52  * sendFileInfo
53  * 用于下载操作：从数据库查询文件信息并发送给客户端。
54  */
55 void FileTrans::sendFileInfo()
56
57 /*
58  * recvFile
59  * 接收上传文件的主流程：
60  * - 先使用 memcached 检查客户端上报的 MD5 是否已存在（秒传逻辑）。
61  * - 若 memcached 未命中，则调用 createFile 从 socket 接收文件内容并
    写入磁盘。
62  * - 计算/记录服务器端的文件 MD5（仅作参考），将文件元信息写入 MySQL
    。
63  * 返回 true 表示函数执行完成（注意：函数内部会将操作结果写入 msg）。
64  */
65 bool FileTrans::recvFile()
66
67 /*
68  * showFileList
69  * 查询指定 `user_id` 的文件列表并以 "id|file_name|file_size\n" 的文
    本格式返回给客户端。
70  * 注意：返回的是纯文本行，客户端需按此协议解析。若无文件则返回 "No
    files"。
```

```

71  */
72 void FileTrans::showFileList()
73
74  /*
75  *  sendFile
76  *  读取本地文件并使用 sendfile 将数据直接传输到客户端套接字 `connfd`
77  *  。
78  *  该实现依赖于内核零拷贝 sendfile，因此效率较高；在失败时会记录错误
79  *  并返回 false。
80  */
81 bool FileTrans::sendFile()
82
83  /*
84  *  createFile
85  *  创建文件并接收上传内容：为用户创建存储目录，从 socket 接收数据并写
86  *  入文件。
87  */
88 bool FileTrans::createFile()
89
90  /*
91  *  main
92  *  程序入口：创建 FileTrans 实例并启动服务。
93  */
94 int main()

```

websocket\_clinet.cpp

```

1  #include "websocket_client.hpp"
2  #include "../common/logger.hpp"
3  #include <iostream>
4  #include <thread>
5
6  namespace chat {
7
8  /**
9   * @brief 构造函数
10  *
11  * 初始化WebSocket客户端，设置事件处理器
12  *
13  * @param username 客户端用户名
14  */

```

```
15 ChatClient::ChatClient(const std::string& username)
16
17 /**
18  * @brief 析构函数
19  *
20  * 确保客户端正确断开连接
21  */
22 ChatClient::~ChatClient()
23
24 /**
25  * @brief 连接到WebSocket服务器
26  *
27  * 在新线程中运行客户端事件循环
28  *
29  * @param uri 服务器URI (格式: ws://host:port)
30  */
31 void ChatClient::connect(const std::string& uri)
32
33 /**
34  * @brief 断开与服务器的连接
35  */
36 void ChatClient::disconnect()
37
38 /**
39  * @brief 发送消息到服务器
40  *
41  * @param message 要发送的消息内容
42  */
43 void ChatClient::send(const std::string& message)
44
45 /**
46  * @brief 设置消息处理回调函数
47  *
48  * @param callback 消息处理函数
49  */
50 void ChatClient::setMessageCallback(std::function<void(const Message
    &)> callback)
51
52 /**
53  * @brief 处理连接建立事件
```

```

54  *
55  * @param hdl 连接句柄
56  */
57 void ChatClient::onOpen(ConnectionHdl hdl)
58
59 /**
60  * @brief 处理连接关闭事件
61  *
62  * @param hdl 连接句柄
63  */
64 void ChatClient::onClose(ConnectionHdl hdl)
65
66 /**
67  * @brief 处理接收到的消息
68  *
69  * 解析消息并调用回调函数
70  *
71  * @param hdl 连接句柄
72  * @param msg 消息内容
73  */
74 void ChatClient::onMessage(ConnectionHdl hdl, WebSocketClient::
    message_ptr msg)
75
76 /**
77  * @brief 运行客户端事件循环
78  *
79  * 处理WebSocket事件
80  */
81 void ChatClient::run()
82
83 } // namespace chat

```

websocket\_server.cpp:

```

1  #include "websocket_server.hpp"
2  #include "../common/logger.hpp"
3  #include <iostream>
4  #include <thread>
5
6  namespace chat
7  {

```

```
8
9  /**
10   * @brief 构造函数
11   *
12   * 初始化WebSocket服务器，设置事件处理器
13   *
14   * @param port 服务器监听端口
15   */
16 ChatServer::ChatServer(uint16_t port)
17
18 /**
19  * @brief 析构函数
20  *
21  * 确保服务器正确关闭
22  */
23 ChatServer::~ChatServer()
24
25 /**
26  * @brief 启动服务器
27  *
28  * 在新线程中运行服务器事件循环
29  */
30 void ChatServer::start()
31
32 /**
33  * @brief 停止服务器
34  *
35  * 关闭所有连接并停止服务器
36  */
37 void ChatServer::stop()
38
39 /**
40  * @brief 广播消息给所有连接的客户端
41  *
42  * @param message 要广播的消息
43  */
44 void ChatServer::broadcast(const std::string &message)
45
46 /**
47  * @brief 设置消息处理回调函数
```

```
48     *
49     * @param callback 消息处理函数
50     */
51 void ChatServer::setMessageCallback(std::function<void(const
    Message &)> callback)
52
53 /**
54     * @brief 处理新的客户端连接
55     *
56     * @param hdl 连接句柄
57     */
58 void ChatServer::onOpen(ConnectionHdl hdl)
59
60 /**
61     * @brief 处理客户端断开连接
62     *
63     * @param hdl 连接句柄
64     */
65 void ChatServer::onClose(ConnectionHdl hdl)
66
67 /**
68     * @brief 处理接收到的消息
69     *
70     * 解析消息并调用回调函数
71     *
72     * @param hdl 连接句柄
73     * @param msg 消息内容
74     */
75 void ChatServer::onMessage(ConnectionHdl hdl, WebSocketServer::
    message_ptr msg)
76
77 /**
78     * @brief 运行服务器事件循环
79     *
80     * 处理WebSocket事件
81     */
82 void ChatServer::run()
83
84 } // namespace chat
```